

算法设计与分析实验报告

胡豪宇 2023302061234

2025 年 5 月

目录

1	Report1	1
1.1	rp++	1
1.1.1	形式化定义	1
1.2	数组游戏	1
1.2.1	形式化定义	1
1.2.2	解题思路	2
1.2.3	复杂度分析	3
1.3	你喜欢 Inf、并查集和算法实验吗	3
1.3.1	形式化定义	3
1.3.2	输入输出要求	3
1.3.3	并查集应用	4
1.3.4	敌对关系检查	4
1.4	Sherway 的有向图	4
1.4.1	形式化定义	4
1.4.2	floyd 改进	5
1.4.3	操作 3 优化	5
1.4.4	复杂度分析	5
2	Report2	6
2.1	回文数组	6
2.1.1	形式化定义	6
2.1.2	解题思路	6
2.2	平衡饮食	6
2.2.1	形式化定义	6
2.2.2	解题思路	6
2.2.3	复杂度分析	7
2.3	其实 Allen 是幕后大 BOSS	7
2.3.1	形式化定义	7
2.3.2	解题思路	7
2.3.3	复杂度分析	8

2.4	Sherway 与幂	8
2.4.1	形式化定义	8
2.4.2	解题思路	8
2.4.3	复杂度分析	8
3	Report3	9
3.1	inf 的旅行计划	9
3.1.1	形式化定义	9
3.1.2	解题思路	9
3.1.3	复杂度分析	10
3.2	最小异或路径	10
3.2.1	形式化定义	10
3.2.2	解题思路	11
3.2.3	复杂度分析	11
3.3	树学长 QwQ 下象棋只会用车	12
3.3.1	形式化定义	12
3.3.2	解题思路	12
3.3.3	复杂度分析	12
3.4	Sherway 的卡牌	12
3.4.1	形式化定义	12
3.4.2	解题思路	13
3.4.3	复杂度分析	14

1 Report1

1.1 rp++

1.1.1 形式化定义

给定字符串 $S = s_1s_2...s_n$ ，其中 $n \leq 2 \times 10^6$ ，定义模式：

$$P = \text{rp}++ = [r][p][+][+]$$

要求统计 P 在 S 中的所有精确匹配次数

1.2 数组游戏

1.2.1 形式化定义

给定初始空数组 $A_0 = \emptyset$ ，经过 q 次操作后得到数组序列 $\{A_t\}_{t=1}^q$ ，定义第 t 次操作后的炫酷值为：

$$F(A_t) = \sum_{i=1}^{|A_t|} A_t[i] \cdot i$$

操作类型集合 $\mathcal{O} = \{1, 2, 3\}$ 定义如下：

- op = 1: 循环右移, $A_t = \text{rotate}(A_{t-1}) = [a_n, a_1, \dots, a_{n-1}]$
- op = 2: 反转数组, $A_t = \text{reverse}(A_{t-1}) = [a_n, \dots, a_1]$
- op = 3 (k): 追加元素, $A_t = A_{t-1} \circ [k]$

对于复杂度:

- 操作次数: $q \leq 2 \times 10^5$
- 追加数值: $k \leq 10^6$
- 时间限制: 通常为 1-2 秒 (需 $O(1)$ 或 $O(\log n)$ 每次操作)

1.2.2 解题思路

本解法采用双端队列 + 标记法高效维护动态数组操作, 核心思想如下:

1. 状态维护:

- 使用双端队列 dq 存储实际数组元素
- 标记位 $reversed$ 记录当前是否处于反转状态
- 维护三个关键变量:

$$S = \sum_{i=1}^n (a_i \times i) \quad (\text{当前炫酷值})$$

$$T = \sum_{i=1}^n a_i \quad (\text{元素总和})$$

$$n = |dq| \quad (\text{数组长度})$$

2. 操作处理 (经数学推导可证明):

- 循环右移 (op=1):

因为:

$$S' = a_n \cdot 1 + \sum_{i=1}^{n-1} a_i \cdot (i+1) = S + \sum_{i=1}^n a_i - n \cdot a_n$$

所以有:

$$S \leftarrow S + \underbrace{T - n \times a_{\text{移动元素}}}_{\text{增量调整}}$$

根据 $reversed$ 状态决定移动首部或尾部元素

- 数组反转 (op=2):

因为:

$$S' = \sum_{i=1}^n a_{n-i+1} \cdot i = \sum_{j=1}^n a_j \cdot (n-j+1) = (n+1)T - S$$

所以有：

$$S \leftarrow (n+1) \times T - S$$

通过数学变换避免实际反转操作

- 追加元素（op=3）：

显然有：

$$\begin{cases} S \leftarrow S + k \times (n+1) \\ T \leftarrow T + k \\ n \leftarrow n + 1 \end{cases}$$

根据 *reversed* 状态决定插入方向

1.2.3 复杂度分析

操作类型	时间复杂度
循环移位	$O(1)$
数组反转	$O(1)$
追加元素	$O(1)$
查询炫酷值	$O(1)$

1.3 你喜欢 Inf、并查集和算法实验吗

1.3.1 形式化定义

给定社交关系图 $G = (V, E_f \cup E_h)$ ，其中：

- V 表示所有熟人节点 ($|V| = n$)
- E_f 为友谊边集（无向边）
- E_h 为敌对边集（无向边）
- 满足 $E_f \cap E_h = \emptyset$

一个合法派对 $P \subseteq V$ 需满足：

1. 朋友闭包性： $\forall v \in P$ ，其所有朋友必须被邀请

$$N_f(v) \subseteq P \quad \text{其中} \quad N_f(v) = \{u \mid (v, u) \in E_f\}$$

2. 无敌对约束：派对内不存在互相敌对的两人

$$\nexists (u, v) \in E_h \quad \text{s.t.} \quad u, v \in P$$

3. 连通性：派对成员在友谊图中连通

$$\forall u, v \in P, \exists \text{路径 } u \leftrightarrow v \text{ 仅通过 } E_f \text{ 中的边}$$

1.3.2 输入输出要求

- 第一行: n (熟人数量, $2 \leq n \leq 2000$)
- 第二行: k (友谊边数, $0 \leq k \leq \min(10^5, \frac{n(n-1)}{2})$)
- 随后 k 行: 每行 $u_i v_i$ 表示一对朋友
- 下一行: m (敌对边数, $0 \leq m \leq \min(10^5, \frac{n(n-1)}{2})$)
- 随后 m 行: 每行 $u_j v_j$ 表示一对敌对关系

寻找满足条件的最大派对人数, 若不存在合法解则输出 0。

输入:

```
9
8
1 2
1 3
2 3
4 5
6 7
7 8
8 9
9 6
2
1 6
7 9
```

- 有效解: $\{1, 2, 3\}$ (大小 3) 和 $\{4, 5\}$ (大小 2)
- 无效解: $\{6, 7, 8, 9\}$ (违反敌对约束), $\{1, 2, 3, 4, 5\}$ (违反连通性)
- 输出: 3

1.3.3 并查集应用

我们注意到输入数据的特点, 每对人最多在输入中被提到一次, 两个人不能同时是朋友和彼此不喜欢, 所以我们可以考虑用并查集来管理不同的朋友链。

而在unite()的过程中, 需要注意维护当前朋友链的大小Size[n], 结合按秩合并的思想, 我们可以用这个数组来指导不同的朋友链之间的合并。

1.3.4 敌对关系检查

接下来我们进行敌对关系检查, 只需要逐一验证每个朋友链中是否存在敌对关系即可, 如果存在, 那么将这个朋友链标记为false即可。

1.4 Sherway 的有向图

1.4.1 形式化定义

给定有向完全图 $G = (V, E)$, 其中:

- 顶点集 $V = \{1, 2, \dots, n\}$
- 边集 E 由邻接矩阵 a 定义, $a_{i,j}$ 表示边 (i, j) 的权重
- 初始所有顶点处于关闭状态 $C = V$, 开放顶点集 $O = \emptyset$

定义状态转移函数:

$$\begin{cases} O \leftarrow O \cup \{x\} & \text{操作 1 x} \\ O \leftarrow O \setminus \{x\} & \text{操作 3 (撤销最近操作 1)} \end{cases}$$

对于操作 2 $x \rightarrow y$, 定义:

$$\delta(x, y) = \min_{\substack{p \in P(x, y) \\ p \cap C = \emptyset}} \sum_{(u, v) \in p} a_{u, v}$$

其中 $P(x, y)$ 为 x 到 y 的所有路径集合。

- 邻接矩阵满足 $a_{i,i} = 0, \forall i \in V$
- 操作约束:
 - 操作 1 前 $x \in C$
 - 操作 2 前 $x, y \in O$
 - 操作 3 前存在未撤销的操作 1
- 空间复杂度: $O(n^2)$ 存储邻接矩阵
- 时间复杂度:
 - 操作 1/3: $O(1)$
 - 操作 2: $O(n^2)$ (Floyd-Warshall 动态更新)

1.4.2 floyd 改进

题目的数据范围限定了用经典的复杂度为 $O(n^3)$ 的算法, 会超时 (只过两个点), 我们要想到, 大部分路线是没必要重新计算的, 新引入一个点带来的影响的路径, **只可能是以这个点为中间结点的路径**。

1.4.3 操作 3 优化

因为操作 3 不是随机取消一个点, 而是撤销上一个操作 1, 因此我们可以**用一个栈来维护被操作 1 更新过的图**, 这样我们就可以不再计算一次 floyd 算法, 提高效率。

当然这里如果不采用 `cur` 数组来维护, 全局只用一个栈, 栈顶表示当前图也是可以的, 这样在空间复杂度不变的情况下, 避免了上面操作 3 的拷贝操作带来的时间损耗, 可以提高运行效率。

操作类型	时间复杂度
打开结点	$O(n^2)$
查询最短路径	$O(1)$
撤销操作 1	$O(n^2)$ (可优化为 $O(1)$)

1.4.4 复杂度分析

所以本题的总体时间复杂度为 $O(t_s \cdot n^2)$, 空间复杂度为 $O(t_1 \cdot n^2)$; 可以优化为时间复杂度为 $O(t_1 \cdot n^2)$, 空间复杂度为 $O(t_1 \cdot n^2)$ 。(t_s 是操作 1,3 的总数量, t_1 是操作 1 的数量)

2 Report2

2.1 回文数组

2.1.1 形式化定义

将一个数字序列经过若干次操作后变成回文序列, 求出将该数组变为回文数组所需的最小操作次数。操作支持如下:

- 选择数组中相邻的两个数, 同时将其加 1 或减 1;
- 选择数组中单独的一个数, 将其加 1 或减 1。

2.1.2 解题思路

联想这道题的解题思路: <https://www.luogu.com.cn/problem/P5019>

我们可以想到贪心的思路, 上面那道题中, 我们可以靠前一个坑顺便填充下一个坑, 这道题我们可以发现第 1 个操作就是一个顺便的操作。所以我们记录好每个位置的对称差, 再让前一个对称差能够顺便填充后一个对称差即可。不过需要注意的是对称差是有正负的, 和坑洞填充只有一个方向是不同的。

2.2 平衡饮食

2.2.1 形式化定义

背包的容量为 X , 其中要装入 3 种物品, 每种物品有其对应的价值 A_i 和体积 C_i , 现在希望你采取合适的装包方式, 使得 3 种物品各自的总价值 F_1, F_2, F_3 的最小值最大。即, 使得 $\min(F_1, F_2, F_3)$ 最大。

2.2.2 解题思路

采用动态规划和二分答案的思想。

- 状态定义: $dp[i][j]$, 种类 i 用 j 容量所能获得的价值最大值
- 状态转移方程: $dp[i][j] = \max(dp[i][j], dp[i][j - c] + a)$, 对每种食物, 需要逆向更新数组, 经典的 01 背包问题。
- 初始条件: $dp[i][0] = 0$, 其他均设置为 INF , 表示该情况非法。

- 前缀最大值优化: $dp[i][j] = \max(dp[i][j], dp[i-1][j])$, 如果当前容量 j 下, 无法获得价值, 则直接取上一步的最大值。

对于本题, 显然是满足单调性的, 如果可以获取更大的最低价值, 则一定可以获取更小的最低价值。所以满足**二分答案**的条件。即, 将优化问题转化为判定问题。对于给定的目标值 mid , 判断是否存在一个组合可以使得在满足容量限制的情况下, 3 种物品的总价值均大于 mid 。这样就能说明答案至少是 mid , 可以尝试更大的答案。

2.2.3 复杂度分析

对于状态转移部分, 每个物品都要遍历 $x - c$ 次, 总共 n 个物品, 所以状态函数计算部分的复杂度为:

$$T_{DP} = O(n \cdot x)$$

二分答案对于验证部分需要对 3 种物品进行二分查找:

$$T_{check} = O(3 \cdot \log x) = O(\log x)$$

设物品中最大的价值为 V_{max} , 则二分的次数为:

$$T_{search} = O(\log V_{max})$$

所以总时间复杂度为:

$$T_{total} = T_{DP} + T_{check} \cdot T_{search} = O(n \cdot x) + O(\log x \cdot \log V_{max})$$

2.3 其实 Allen 是幕后大 BOSS

2.3.1 形式化定义

给定一个长度为 n (n 为奇数) 的员工序列, 其中第 i 个员工的工资范围为 $[l_i, r_i]$ 。初始总工资满足 $\sum_{i=1}^n l_i \leq s$ 。我们需要为每个员工分配工资 $w_i \in [l_i, r_i]$, 满足:

1. 总工资约束: $\sum_{i=1}^n w_i \leq s$
2. 最大化工资序列的中位数

设 $k = \frac{n+1}{2}$, 优化目标为:

$$\max \{ \text{median}(w_1, w_2, \dots, w_n) \}$$

约束条件:

$$\begin{cases} l_i \leq w_i \leq r_i & \forall i \in \{1, 2, \dots, n\} \\ \sum_{i=1}^n w_i \leq s \end{cases}$$

2.3.2 解题思路

先发出基础工资，把每位员工工资初始化为 $l[i]$ ，再看此时高于设定目标中位数 mid 的员工数量，若达到要求就返回，否则调整后续员工工资；为了调整中位数但尽可能小的增加总工资，应该优先调整**最接近中位数的元素**，所以综合考虑前面的论述，我们必须要对员工的最低工资 $l[i]$ 进行**从大到小**排序。

当目标中位数 mid 可以达成时，则挑战更高的中位数标准，若不行，则尝试更低的中位数标准。需要注意的是，每次尝试之间需要把总工资回调到原来的状态，这里我们并没有对每个人的具体工资做改动，而是只改动总工资，在效果上相当于改动了每个人的具体工资。

2.3.3 复杂度分析

- 预处理: $O(n)$ 计算初始总工资
- 排序: $O(n \log n)$
- 二分查找: $O(\log(\max r_i))$
- 每次检查: $O(n)$
- 总复杂度: $O(n \log n + n \log(\max r_i))$

2.4 Sherway 与幂

2.4.1 形式化定义

求 1 到 N 中共有多少个数可以表示成 $M^K, K > 1, N \leq 1e18$.

2.4.2 解题思路

我们会先有一个朴素的思想，也就是我对 n 进行开根，把开根的结果取整后加起来：

$$ans = \sum_{i=2}^{\infty} \lfloor (n)^{\frac{1}{i}} \rfloor$$

但是这样我们发现，会对个别项重复计算，例如 $2^6 = 4^3 = 8^2 = 64$ 被算了 3 次，我们应该对其进行去重。这样我们就应该联想到容斥原理来确保不会重复计算，也不会额外扣除。

设 A_i 表示所有 $p[i]$ 次方数的集合 ($p[i]$ 表示第 i 个质数)，即：

$$A_i = \{x^{p[i]} \leq n | x \in \mathbb{N}\}$$

我们需要计算的就是 $|\bigcup_{i=1}^{\infty} A_i|$ ，即所有次方数的并集。所以有：

$$|\bigcup_{i=1}^{\infty} A_i| = \sum_{i=1}^{\infty} |A_i| - \sum_{1 \leq i < j} |A_i \cup A_j| + \sum_{1 \leq i < j < k} |A_i \cup A_j \cup A_k| \dots$$

为了计算上式，所以我们首先需要确定指数取值的上界，只需要找到 $imax$ 使得 $2^{p[imax]} \leq n < 2^{p[imax+1]}$ 即可。接着在中间的计算过程中，对于由**奇数**个质数相乘得到的指数幂集合应该**加上**，而对于**偶数**质数相乘得到的指数幂集合应该**减去**。这也都是我们需要统计的。

需要额外注意的是，这里我们可以利用已求出的 $imax$ ，那么从 1 到 2^{imax} 中的每一个 j ，它的二进制表示可以算作是对于素数数组的一个选取规则。例如 0b101 可以表示选择了素数 2 和 5。这样只需要记录其中 1 的数量，就知道其选取的素数的奇偶数了。

2.4.3 复杂度分析

- 外层循环: $2^{imax} - 1$ 次
- 内层循环 $imax$ 次
- 总复杂度 $O(imax \cdot (2^{imax} - 1))$ (其中 $imax$ 约为 2 到 $\log n$ 中质数的数量, 在本题中不超过 20)

3 Report3

3.1 inf 的旅行计划

3.1.1 形式化定义

有两个城市, 分别有 n_1 和 n_2 个小镇, 以及 m_1 和 m_2 条路径。每条路径连接同一城市中的两个小镇, 经过路径需要消耗 x_i 的油费。同一城市的所有小镇通过路径连通。旅行者 INF 住在城市 1 的 1 号小镇, 计划前往城市 2 的 1 号小镇。现需在两个城市中各选一个小镇建立一条新路径, 消耗油费为 k 。由于无法预知具体选择的小镇, 需计算保证到达目的地所需的最少油费。

输入格式 第一行包含五个正整数 n_1, n_2, m_1, m_2, k , 分别表示:

- 城市 1 的小镇数量 n_1 和路径数量 m_1 ,
- 城市 2 的小镇数量 n_2 和路径数量 m_2 ,
- 新建路径的油费 k 。

接下来 m_1 行, 每行三个正整数 u_{i1}, v_{i1}, x_{i1} , 表示城市 1 中小镇 u_{i1} 和 v_{i1} 之间存在一条路径, 油费为 x_{i1} 。

随后 m_2 行, 每行三个正整数 u_{i2}, v_{i2}, x_{i2} , 表示城市 2 中小镇 u_{i2} 和 v_{i2} 之间存在一条路径, 油费为 x_{i2} 。

输出格式 一行一个整数, 表示保证到达城市 2 的 1 号小镇所需的最少油费。

3.1.2 解题思路

给定两个城市的图结构, 我们需要在两个城市中各选一个节点建立一条新边, 使得从城市 1 的起点到城市 2 的起点的路径油费最小化 (在最坏情况下)。以下是解题步骤:

1. 图结构存储与预处理

- 使用邻接表存储两个城市的图结构 (城市 1 和城市 2 分别用 `type=0` 和 `type=1` 区分)。
- 每条边是双向的, 因此在输入时需添加正向和反向边。

2. 计算单源最短路径

- 使用 Dijkstra 算法分别计算：
 - 城市 1 中从起点 1 到所有其他节点的最短油费 $\text{dist}[0][i]$ 。
 - 城市 2 中从起点 1 到所有其他节点的最短油费 $\text{dist}[1][i]$ 。
- 由于所有边权均为正，Dijkstra 算法可以高效求解。

3. 分析最坏情况下的油费

- 新建边的油费为 k ，因此总油费为：

$$\text{Cost}(u, v) = \text{dist}[0][u] + k + \text{dist}[1][v]$$

- 由于无法预知具体选择的 u 和 v ，需考虑最坏情况，即：

$$\text{Answer} = \max_{u \in S_1, v \in S_2} (\text{dist}[0][u] + k + \text{dist}[1][v])$$

- 为了最小化该最大值，应选择使得 $\text{dist}[0][u]$ 和 $\text{dist}[1][v]$ 尽可能小的 u 和 v 。

4. 求解最小化最大油费

- 实际上，最坏情况发生在：
 - 城市 1 中选择离起点 1 最远的节点 u （即 $\text{dist}[0][u]$ 最大）。
 - 城市 2 中选择离起点 1 最远的节点 v （即 $\text{dist}[1][v]$ 最大）。
- 因此，最终答案为：

$$\text{Answer} = \max_{2 \leq i \leq n_1} \text{dist}[0][i] + k + \max_{2 \leq j \leq n_2} \text{dist}[1][j]$$

- 代码中通过遍历 $\text{dist}[0]$ 和 $\text{dist}[1]$ 数组找到最大值并相加得到结果。

3.1.3 复杂度分析

- Dijkstra 算法的时间复杂度为 $O(m \log n)$ ，其中 m 是边数， n 是节点数。
- 由于对两个城市分别运行一次 Dijkstra，总时间复杂度为 $O((m_1 + m_2) \log(n_1 + n_2))$ 。
- 后续遍历求最大值的时间复杂度为 $O(n_1 + n_2)$ ，可以忽略。

3.2 最小异或路径

3.2.1 形式化定义

给定一个简单无向连通图 $G = (V, E)$ ，其中：

- 节点集合 $V = \{1, 2, \dots, n\}$ ， $|V| = n$ ；

- 边集合 E , $|E| = m$, 每条边 $e = (u_i, v_i) \in E$ 具有边权 w_i 。

定义从节点 1 到节点 n 的简单路径 P 的异或和为:

$$\bigoplus_{(u,v) \in P} w_{(u,v)}$$

其中:

- 简单路径: 路径不重复经过任何节点;
- 异或和 ($\oplus$): 路径上所有边权的按位异或结果。

目标 在所有可能的简单路径 P 中, 找到使得异或和最小的路径, 并输出该最小异或和:

$$\text{Answer} = \min_{P \in \mathcal{P}(1,n)} \left(\bigoplus_{(u,v) \in P} w_{(u,v)} \right)$$

其中 $\mathcal{P}(1,n)$ 表示所有从节点 1 到节点 n 的简单路径集合。

3.2.2 解题思路

给定一个简单无向连通图, 要求找到从节点 1 到节点 n 的所有简单路径中边权异或和最小的路径。以下是基于深度优先搜索 (DFS) 的解题步骤:

1. 图表示与初始化

- 使用邻接表存储图结构, 其中每个节点保存其相邻节点及对应边权。
- 初始化全局变量:
 - `min_xor`: 记录当前最小异或和, 初始设为最大值 (`INF`)。
 - `vis`: 标记节点是否被访问过的数组, 初始为未访问 (`false`)。

2. 深度优先搜索 (DFS)

- 从起点 1 开始递归搜索, 维护当前路径的异或和 `cur_xor`。
- 递归终止条件: 到达目标节点 n 时, 更新 `min_xor`:

$$\text{min_xor} = \min(\text{min_xor}, \text{cur_xor})$$

- 对于当前节点 u 的每个未访问邻居 v :
 - 标记 v 为已访问。
 - 递归计算 v 到 n 的路径异或和: `cur_xor  $\oplus$   $w$` (w 为边权)。
 - 回溯时恢复 v 的未访问状态, 确保其他路径可探索。

3. 算法正确性

- 简单路径约束：通过 `vis` 数组避免重复访问节点，保证路径简单性。
- 全局最优性：DFS 遍历所有可能的简单路径，通过比较 `cur_xor` 确保找到最小异或和。
- 连通性保证：题目给定的图连通，确保至少存在一条从 1 到 n 的路径。

3.2.3 复杂度分析

- 最坏情况下需遍历所有简单路径，时间复杂度为 $O((n-1)!)$ （如完全图）。
- 实际应用中，可通过剪枝优化（如提前终止大于当前 `min_xor` 的路径），但无法改变指数级复杂度。

3.3 树学长 QwQ 下象棋只会用车

3.3.1 形式化定义

在阶梯形地图中放置 k 个车，使得他们不会互相攻击的方案数汇总。

3.3.2 解题思路

1. 预处理阶乘和逆元：

- 使用快速幂算法预先计算阶乘数组 `fact` 和逆阶乘数组 `inv_fact`，模数为 100003。
- 逆元通过费马小定理计算： $a^{-1} \equiv a^{p-2} \pmod{p}$ 。

2. 组合数计算：

- 函数 $C(n, m)$ 利用预处理的阶乘和逆元计算组合数 $\binom{n}{m} \bmod 100003$ 。
- 处理边界情况：当 $m > n$ 或 $m < 0$ 时返回 1。

3. 主计算逻辑：

- 遍历所有可能的分配方式：将 k 分为 i （分配给 c 和 d 类）和 $j = k - i$ （分配给其他类）。
- 约束条件：

- $i \leq \min(c, d, k)$
- $j = k - i \geq 0$
- $a + e \geq j$ （确保有足够容量）

- 对每个有效分配计算组合数乘积：

$$\binom{b+d-i}{j} \times \binom{a+e}{j} \times j! \times \binom{c}{i} \times \binom{d}{i} \times i!$$

- 将各项相乘并对结果取模。

4. 输出结果：

- 累加所有有效分配方式的结果，输出最终值 $\bmod 100003$ 。

3.3.3 复杂度分析

时间复杂度约为 $O(MAXN)$

3.4 Sherway 的卡牌

3.4.1 形式化定义

给定初始状态 $S_0 = \{1\}$ （仅含一张攻击力为 1 的卡牌）和事件序列 $A = (a_1, a_2, \dots, a_n)$ ，其中 $\forall a_i \in \{-1, 0, 1\}$ 。定义状态转移规则如下：

- 拿牌事件 ($a_i = 1$):

若当前状态为 $S_{i-1} = \{x_1, x_2, \dots, x_m\}$ ，则新状态为 $S_i = S_{i-1} \cup \{1\}$ ，其中 $m \geq 0$ 。

- 弃牌事件 ($a_i = -1$):

若 $|S_{i-1}| \geq 2$ ，选择任意 $x_u, x_v \in S_{i-1}$ ，转移至 $S_i = (S_{i-1} \setminus \{x_u, x_v\}) \cup \{x_u + x_v\}$ ；否则游戏失败（输出 -1）。

- 抽奖事件 ($a_i = 0$):

可选择执行 $a_i = 1$ 或 $a_i = -1$ 的转移规则（必须选择其一）。

定义最终平均攻击力为：

$$\text{Avg}(S_n) = \frac{1}{|S_n|} \sum_{x \in S_n} x$$

输入：

第一行为 $n \in \mathbb{Z}^+$ ，第二行为序列 $A = (a_1, \dots, a_n)$ 。

输出：

若存在转移路径使得 $\forall i \leq n, |S_i| \geq 1$ ，则输出 $\max \text{Avg}(S_n)$ 的最简分数形式 $\frac{p}{q}$ （需满足 $\gcd(p, q) = 1$ ）；否则输出 -1。

3.4.2 解题思路

通过动态维护卡牌数量和攻击力总和，结合贪心策略确保事件序列可行并最大化平均攻击力。

1. 预处理阶段（逆向扫描）：

- 定义数组 `req_min`，其中 `req_min[i]` 表示处理完第 i 个事件后所需的最小卡牌数
- 初始化 `req_min[n] = 1`（最终至少保留 1 张牌）
- 逆向遍历事件序列，根据事件类型更新最小需求：
 - 弃牌事件 ($a_i = -1$): 至少需要 $\max(2, \text{req_min}[i+1] + 1)$ 张牌
 - 拿牌事件 ($a_i = 1$): 至少需要 $\max(1, \text{req_min}[i+1] - 1)$ 张牌
 - 抽奖事件 ($a_i = 0$): 取拿牌和弃牌需求的最小值

2. 可行性判断：

- 若初始需求 `req_min[0] > 1`，说明无法完成所有事件，直接输出 -1

3. 正向处理阶段:

- 初始化当前卡牌数 $k = 1$ 和攻击力总和 $S = 1$
- 遍历事件序列动态更新状态:
 - 拿牌事件: k 和 S 均增加 1
 - 弃牌事件: k 减少 1 (攻击力总和不变)
 - 抽奖事件: 选择使 k 尽可能小的操作 (贪心策略):
 - * 若 $k - 1 \geq \text{req_min}[i+1]$ 则弃牌
 - * 否则拿牌

4. 结果计算:

- 计算 S 与 k 的最大公约数 $g = \text{gcd}(S, k)$
- 输出最简分数形式 $\frac{S/g}{k/g}$

3.4.3 复杂度分析

时间复杂度 $O(n)$ (预处理和正向处理各扫描一次)